

# LOGBOOK OF FMU\_explore

## DEVELOPMENT

Here I describe the status of development the module FMU\_explore. The code has been developed over long time as a gernal part of a setup script for BPL applications. Now the time has come to encapsulate this code as a central module. The simplified FMU interaction can be used for Modelica simulation models in general and not just BPL applications.

This informal text briefly describes the status and goals in near time. The idea is that this text here will complement the formal git-version handling. Now and then I plan to introduce reference to the git-version commit history.

Key architecture features are:

- Class structure of the module
- The module is then adapted to an applications through a number of variables
- The module placed in a package to simplify future use both locally and in Github

Development is mainly done in the following files:

- fmu\_explore\_pyfmi.py
- fmu\_explore\_fmipy.py
- setup TEST2\_Batch.py
- setup TEST2\_Batch\_explore.py

I think the different examples serve as a good test bench for changes in the library. The collection of processes also is a good starting point when new systems are developed.

A general goal is:

Specific goals - now reached:

- Module v1 without simu(), show(), setLines() / Done 260819
- Module v2 complete a new concept axes as a list / Done 260819
- Module v2 now works in Windows with PyFMI / Done 260820
- Module v2 now works in Linux with PyFMI and no surprises / Done 260821
- Module v2 stored in package and used both locally and in Colab / Done 260824
- How do I make the performance function cstrProdMax() available inside simu() / Done

260825

- Make `sim_res` available in the Jupyter/Colab notebook when needed / Done 260829
- Make `simulationTime` default for `simu()` and also possible to set in `simu()` / Done 260901
- Make options in `simu()` possible to set / Done 260901
- Make `cstrProdMax()` available during plotting / Done 260901
- Make `profile()` available during plotting / Done 260901
- Fix `simu('cont')` for `fmU_explore_fmpy` and `model_get()` / Done 260907
- Make declaration of version in only one place from today's two places / Done 260909
- Made `prevFinalTime` local to the `FMU_explore` class / Done 260909
- Added to `simu()` the context parameter with `model_get()` good for `eval()` / Done 260909
- Added to `simu()` in `fmU_explore_fmpy` a return of `sim_res` / Done 260910
- Make version of `scipy.optimize` visible in `system_info()` / Done 260910
- Reorganise the list of arguments for the class (and update notebooks) / Done 260911
- 
- Handle `cstrProdMax` etc for `fmpy`
- Test if sufficient to create the model just inside the module

Tentative release note for version 1.3.0 say

Improved code quality using pylint:

- import of packages so that internal packages are first
- import only packages used
- change `type()` to `isinstance()`
- docstrings for class and functions
- change indentation to 4 spaces

## Day to day notes in reversed time order

---

Note that commit ID are now given after the date and description (i.e. opposite to Git). Far from all commits are described here though.

2026-09-18 Started the work yesterday with pylint to improve code quality and adopt to standards. Now call ver **1.2.2**. So far:

- import of packages not used taken away, also adjust setup-files of applications
- import order of packages, third party later
- class docstring
- change `type()` to `isinstance()` - recommended since faster

Later I will change indentation to 4 spaces instead of 3 and make that a separate commit.

2026-09-16 I introduced ver **1.2.1** for maintenance. First thing was to bring back the

possibility to get out the version number by module. **\_version\_**. Before that I briefly checked the need for diagrams, ax, and lines in the setup file and found that they are needed for newplot() communicating with the module functions.

2026-09-15 All applications with adjusted notebooks and run with the released version 1.2.0.

2026-09-14 Now moved definition of statValue from setup to init-file of fmu\_explore\_pyfmi. Polished both modules a little bit. Now call this **ver 1.2.0**. If works in Windows I will make it my first "release" on Github!

2026-09-11 For fmu\_explore\_fmipy I brought in calculation of stateValue to the module from the setup-file. Further I changed the class name to AdaptWith and reorganized the parameters. Call this **ver 1.1.8**

I test applications in Windows locally before I upload to Github and likely done on Monday if everything else works fine.

2026-09-09 Now also BPL\_IEC\_validation works with fmuexplore\_fmipy after a few fixes. But I keep the ver 1.1.6. In the notebook I have started to call the populated FMU\_explore for Adapted instead of Dummy. Later I think I will change the name in the module to say AdaptWith to better bring the understanding of the workflow to the reader of the notebook.

2026-09-08 Fixed in fmu\_explore\_fmipy/decribe\_general() a model.get to model\_get. Call it now **ver 1.1.6**.

2026-09-08 Now BPL\_TEST2\_Fedbatch and BPL\_YEAST\_AIR\_Fedbatch\_DOcontrol works with fmipy in ver 1.1.5.

2026-09-07 Now BPL\_TEST2\_Batch works with fmuexplore\_fmipy in **ver 1.1.5**. This version of FMU\_explore will be uploaded to Github.

2026-09-02 Start the work with fmu\_explore\_fmipy and call it ver 1.1.5

2026-09-01 Brought in the last three applications Chemostat, Perfusion and IEC\_validation and using **FMU\_explore ver 1.1.4**.

2026-09-01 Brought in BPL\_TEST2\_Batch\_calibration.

2026-09-01 I made simu() have parameters simulationTime and options free to set but default taken from the class. Call it **FMU\_explorer ver 1.1.3**.

2026-08-28 Now brought in BPL\_YEAST\_COB\_Batch and BPL\_TEST2\_Batch\_design\_space using FMU\_explore ver 1.1.2

2026-08-29 I made sim\_res available outside simu() by just introducing a return statement in the function. Call this **FMU\_explorer ver 1.1.2**.

2026-08-28 Now brought in BPL\_IEC\_Operation, BPL\_STEM\_AIR\_Perfusion.

2026-08-27 Now brought in BPL\_CHO\_Perfusioncspropenloop.

2026-08-26 Now brought three more applications to Colab: PL\_TEST2\_Chemostat, BPL\_YEAST\_AIR\_Fedbatch, BPL\_CHO\_Fedbatch.

2026-08-25 Almost managed to convert BPL\_TEST2\_Chemostat for FMU\_explore 1.1.0. Remains to handle cstrProdMax() defined in user functions, though.

2026-08-24 The package updated to include import of pandas. Then run locally with BPL\_TEST2\_Batch, BPL\_TEST2\_Fedbatch and BPL\_CHO\_Fedbatch in Windows. The next step is to run these applications also in Colab after updating the Colab notebook. Problem today with pyfmi more general.

2026-08-21 I tested the application BPL\_TEST2\_Batch in Linux and worked fine. Then I made it public at Github Colab and published a note as well.

2026-08-20 I decided to let the itertool work from the module only. The function resetPen() can be used in the function newplot() in the applications script and that is the key. Also the sim\_res file is for the time being just kept global from the module.

2026-08-19 Managed to get v2 working reasonably. The key was to introduce an empty list ax, similar to diagrams, as part of the class. Then newplot() gives both of them appropriate values. Remains to fix setLines() and also simu() bring back start and final simulation time.